

Inhaltsverzeichnis

1	Einleitung	2
2	Aufgabe 1: Schmucknachrichten	2
2.1	Einleitung	2
2.2	Theoretische Erklärung des Algorithmus	3
2.2.1	Betrachtung von Symbolen mit gleichen Kosten	3
2.2.2	Erweiterung auf Symbolen mit unterschiedlichen Kosten	5
2.3	Implementierung des Algorithmus	6
2.3.1	Erstellung des Baums	7
2.3.2	Permutation des Baums	8
2.4	Laufzeitanalyse und Einordnung des Algorithmus	9
2.4.1	Analyse des Algorithmus	9
2.4.2	Stand der Forschung	10
2.4.3	Fazit	11
2.5	Vergleich mit der Musterlösung	11
3	Aufgabe 2: Simultane Labyrinth	12
3.1	Einleitung	12
3.2	Theoretische Erklärung des Algorithmus	12
3.2.1	Erstellung der Matrix	13
3.2.2	Lösen der Labyrinth	13
3.3	Implementierung des Algorithmus	14
3.3.1	Erstellung der Matrix	14
3.3.2	Lösen der Labyrinth	15
3.4	Laufzeitanalyse	16
3.5	Vergleich mit der Musterlösung	18
4	Fazit	19
5	Anhang	20
5.1	Lösungen der Testfälle	20
5.1.1	Aufgabe 1: Schmucknachrichten	20
5.1.2	Aufgabe 2: Simultane Labyrinth	21
5.2	Quellcode	21
5.2.1	Aufgabe 1: Schmucknachrichten	21
5.2.2	Aufgabe 2: Simultane Labyrinth	24
5.3	Quellen	28

1 Einleitung

Die vorliegende Arbeit ist meine Einreichung für die zweiten Runde des 43. Bundeswettbewerb Informatik (BwInf). Es sind zwei Aufgabe zu bearbeiten, die jeweils ein Problem beschreiben. Für jedes Problem soll Algorithmus entwickelt werden, mit dem sich dieses lösen lässt. Anschließend muss der Algorithmus auf verschiedene Testfälle angewandt werden, damit die Korrektoren die Richtigkeit und Genauigkeit beurteilen können. Die Teilnehmenden kennen dabei die erwarteten Ergebnisse der Testfälle nicht. Die ausgearbeitet Lösungen werden anschließend nach einem festgelegten Schema korrigiert und benotet. Pro Aufgabe können bis zu 20 Punkte erreicht werden, wobei gute Ausarbeitungen Zusatzpunkte geben können und unvollständige Bearbeitungen Punktabzüge nach sich ziehen. [1]

Meine Arbeit ist in zwei Teile aufgeteilt, beginnend mit Aufgabe 1, Schmucknachrichten, und dann Aufgabe 2, simultane Labyrinth.

Zuerst erkläre ich jeweils kurz die Aufgabestellung. Danach beschreibe ich meine theoretische Überlegung und zeige im Anschluss, wie ich sie praktisch in Python3 umgesetzt habe. Anschließend diskutiere ich meine Ergebnisse, ziehe ein Fazit und vergleiche schließlich meine Lösung mit der offiziellen Musterlösung.

2 Aufgabe 1: Schmucknachrichten

2.1 Einleitung

In dieser Aufgabe möchten Sybille und Pedram kurze Nachrichten, die aus verschiedenen Buchstaben bestehen, in Perlen-Codes für Ketten umwandeln. Für jeden Buchstaben ist ein Codewort aus Perlen zu finden, wobei insgesamt ein präfixfreier Code entstehen muss.[1] Dabei soll die Gesamtlänge der codierten Nachricht möglichst gering sein. Daher wird ein präfixfreier Code gesucht, bei dem häufig vorkommende Buchstaben kurze Codewörter erhalten, während seltenere Buchstaben längere Codewörter zugewiesen bekommen. „Ein Code C heißt Präfix-Code (irreduzibler Code), wenn kein Codewort aus C Präfix eines anderen Codeworts von C ist.“[7] Dadurch lässt sich jede Nachricht eindeutig codieren und wieder decodieren.[7] Ein Code mit den Codewörtern $\{0, 10, 11\}$ ist präfixfrei, da kein Codewort der Anfang eines anderen ist. Im Gegensatz dazu ist der Code $\{0, 01, 11\}$ nicht präfixfrei ist, weil „0“ der Präfix von „01“ ist. Zusätzlich wird in dieser Aufgabe zwischen zwei Varianten unterschieden: In der ersten Teilaufgabe haben alle Perlen denselben Durchmesser, während in der zweiten Teilaufgabe die Perlen unterschiedliche Durchmesser besitzen. [1]

2.2 Theoretische Erklärung des Algorithmus

2.2.1 Betrachtung von Symbolen mit gleichen Kosten

In meiner Arbeit gilt die Annahme, solange alle Perlen im Durchmesser gleich sind:

$$c_i = p_i \cdot n_i \quad (1)$$

$$l = \sum_i c_i \quad (2)$$

wobei gilt:

- c_i die Kosten des Buchstaben i
- p_i die Häufigkeit des Buchstabens i in der Nachricht
- n_i die Länge des Codewortes für den Buchstaben i (Anzahl der Perlen)
- l die Länge der codierten Nachricht

Um einen präfixfreien Code zu erzeugen, wird üblicherweise ein Baum verwendet. In diesem entspricht jedes Blatt einem Codewort. Die Kanten auf dem Weg von der Wurzel zu einem Blatt repräsentieren dabei die Symbole des jeweiligen Codewortes. Diese präfixfreie Eigenschaft ergibt sich automatisch aus der Baumstruktur. Das heißt, sobald ein Blatt erreicht wird, endet das Codewort, und keine weiteren Knoten bzw. Symbole können daran angeschlossen werden. Der Baum hat mindestens so viele Blätter wie es verschiedene Buchstaben gibt, da jeder Buchstabe sein eigenes Codewort braucht. Jeder innere Knoten kann dabei maximal so viele Kinder haben, wie es verschiedene Perlen gibt. Die Ebene des Blatts entspricht der Länge des Codeworts, also n .

Jeder präfixfreie Code kann in einem Baum dargestellt werden und jeder Baum kann einen präfixfreien Code darstellen.[7] Daher kann das Problem auf ein Baum reduziert werden.

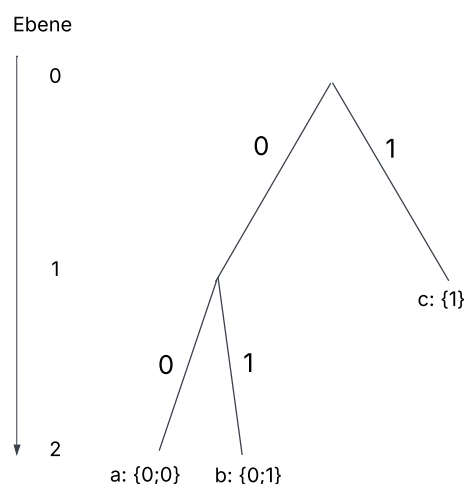


Abbildung 1: einfacher Baum

Um einen möglichst kleinen präfixfreien Code zu erstellen, muss der Buchstabe, der am häufigsten vorkommt, ganz oben eingeordnet werden und die nächsten Buchstaben dann in absteigender Reihenfolge. In Abbildung 1 hätte c die höchste Häufigkeit und danach kommt a und b. Ob der Buchstabe a oder b die höhere Häufigkeit besitzt, ist nicht relevant, da beide auf der untersten Ebene sind.

Daher ergibt sich $p_1 \geq p_2 \geq p_3 \dots \geq p_i$ und $n_1 \leq n_2 \leq n_3 \dots \leq n_i$

Zu finden ist also ein Baum, der basierend auf den Häufigkeiten der Buchstaben, seine Blätter so auf seine Ebenen verteilt, dass die Länge der codierten Nachricht (Gleichung (2)) möglichst gering ist. Jeder innere Knoten muss dabei mindestens zwei Kinder haben, da ein innerer Knoten mit nur einem Kind die Zweige des Baums verlängert (höheres n) ohne einen Vorteile zu bringen.

Zu Beginn des Algorithmus wird ein balancierter Baum erstellt, der so viele Blätter wie Buchstaben hat (Abbildung 2). Jedes Blatt wird von links nach rechts in aufsteigender Reihenfolge der Häufigkeit, mit einem Buchstaben belegt, und die Gesamtkosten des Baums werden gemäß der Formel (2) berechnet. Dabei ergeben sich deutlich höhere Kosten für die rechten Buchstaben im Baum im Vergleich zu den linken, da alle Blätter auf derselben Ebene sind, jedoch die Häufigkeit der Buchstaben auf der rechten Seite größer ist. Um dies auszugleichen, wird der Buchstabe mit den höchsten Kosten eine Ebene nach oben verschoben, indem der Elternknoten des Blattes selber zum Blatt wird und alle Kinder verliert. Dadurch verliert der Baum aber zu viele Kinder, weshalb er erweitert werden muss. Für die Erweiterung untersucht der Algorithmus, an welche Knoten neue Blätter angehängt werden können. Er überprüft die Blätter nach ihrer Ebene, d.h. nach ihrem Wert für n , und fügt das mit dem kleinsten Wert hinzu. Das nach oben verschobenene Blatt erhält den Zusatz *protected*. Das verhindert, dass es selbst weitere Kinder bekommen kann. Dadurch wird ein Zyklus vermieden, bei dem ein Blatt zunächst nach oben verschoben und anschließend wieder nach unten erweitert würde. Nach jeder Iteration werden wie schon zu Beginn den Blättern Buchstaben aufsteigend zugewiesen. Durch die Neuuzuweisung ist sicher gestellt, dass immer die äußerst linken Blätter, die Buchstaben mit den geringsten Häufigkeiten kriegen und rechts die höchsten.

Der Vorgang, ein Blatt nach oben zu schieben und unten neue Blätter hinzufügen, wird so lange fortgesetzt, bis keine neuen Blätter mehr eingefügt werden können, da alle Blätter als *protected* gelten. Das ist der Fall kurz vor einem maximal ausgearteten Baum (Abbildung 5). Der Algorithmus geht also von einem Maximum (maximal ausbalanciert) zum anderen Maximum (maximal ausgeartet). Am Ende wird der Baum mit den geringsten Gesamtkosten ausgegeben.

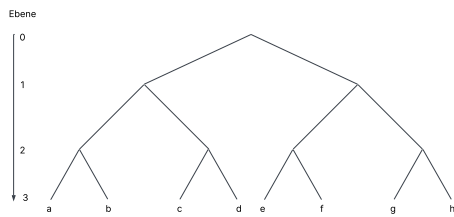


Abbildung 2: Balancierter Baum

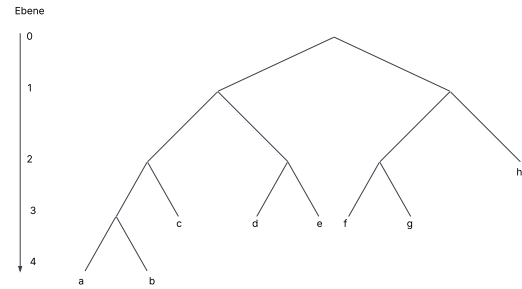


Abbildung 3: Transformation 1

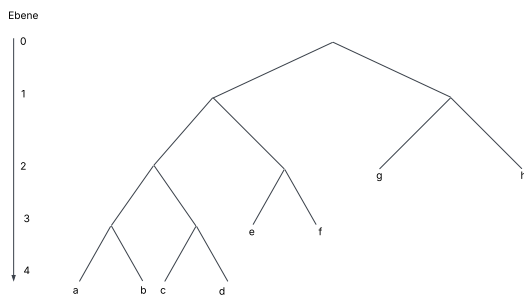


Abbildung 4: Transformation 2

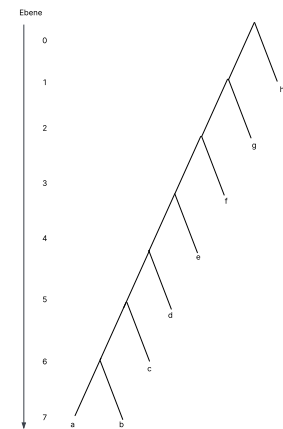


Abbildung 5: Ausgearteter Baum

2.2.2 Erweiterung auf Symbolen mit unterschiedlichen Kosten

Das Problem bei unterschiedlichen Perlendurchmessern besteht darin, dass Blätter auf derselben Ebene unterschiedliche Kosten für die Gesamtlänge der Nachricht haben können. Daher entsprechen die Kosten nicht mehr der Wert der Ebene. Das hat zur Folge, dass die Gleichung (1) nicht mehr stimmt, stattdessen muss der Durchmesser jeder Perle im Codewort einzeln angeschaut werden. Daher ergibt sich

$$c_i = p_i \cdot \sum_j cp_j \quad (3)$$

wobei gilt:

- cp_j die Kosten/Durchmesser der Perle j

Stellt man nun Bäume nicht nach ihrer Ebene, sondern nach ihren Kosten dar, erhält man einen *lopsided tree*[4] (Abbildung 6).

In diesem Fall gilt ebenfalls, jeder innere Knoten muss mindestens zwei Kinder haben. Bei dieser Art von Bäumen hat jedoch nicht der äußerst rechte Knoten die geringsten

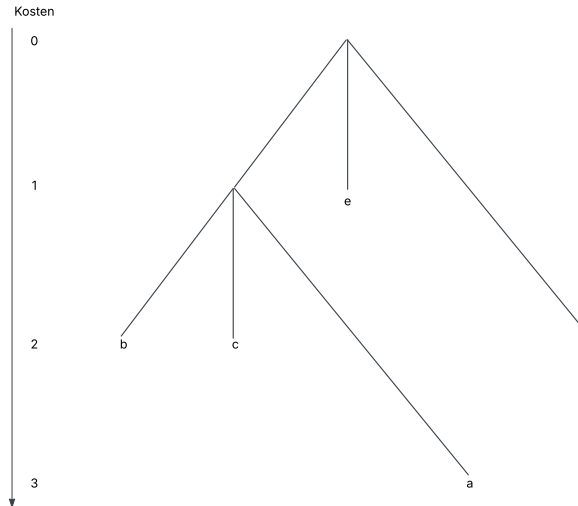


Abbildung 6: lopsided tree: Perlendurchmesser: (1; 1; 2)

Kosten, sondern der oberste. Der Algorithmus bleibt unverändert, d.h. zunächst wird ein Baum erstellt, in dem alle Blätter ungefähr die gleichen Kosten haben. Anschließend wird der Buchstabe mit den höchsten Kosten nach oben verschoben, während der Baum unten erweitert wird. Dies geschieht solange, bis kein Blatt mehr eingefügt werden kann und der beste Baum wird ausgegeben.

Wenn die Durchmesser alle gleich groß sind, gilt:

$$p_i \cdot n_i = p_i \cdot \sum_n cp_n \quad (4)$$

daher kann der zweite Algorithmus sowohl für gleiche als auch unterschiedliche Durchmesser benutzt werden.

2.3 Implementierung des Algorithmus

Das Baum ist über die Klasse *TreeNode* rekursiv implementieren. Jeder Knoten entspricht einem *TreeNode*-Objekt.

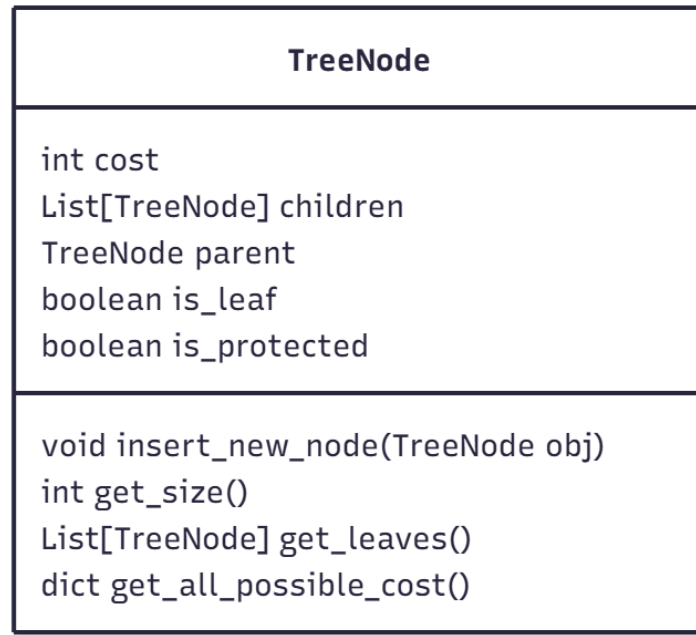


Abbildung 7: Klasse TreeNode

```
1 def get_all_possible_costs(self) -> dict:
```

Listing 1: Methode `get_all_possible_costs`

Die Methode *get_all_possible_costs* prüft, ob das `TreeNode`-Objekt noch weitere Kinder aufnehmen kann und nicht `protected` ist. Falls ja, fügt es sich selbst sowie die Kosten des potenziellen neuen Kindes einem Dictionary hinzu. Anschließend wird die Methode rekursiv für alle bestehenden Kinder aufgerufen, so dass auch deren mögliche Erweiterungen im Dictionary erfasst werden.

```
1 def insert_new_node(self, obj: TreeNode):
```

Listing 2: Methode `insert_new_node`

Die Methode *insert_new_node* prüft, ob das aktuelle Objekt selbst das gesuchte `TreeNode` Objekt ist. Falls ja, fügt es ein neues Kind hinzu. Wenn vorher noch keine Kinder vorhanden sind, werden zwei Kinder hinzugefügt. Ansonsten wird die Methode rekursiv auf alle bestehenden Kinder angewandt.

2.3.1 Erstellung des Baums

Zu Beginn wird ein blancierter Baum über die Methode *create_tree()* erstellt.

```
1 def create_tree(root: TreeNode) -> TreeNode:
```

Listing 3: Methode `create_tree`

Die Methode `create_tree` bekommt über den Aufruf `get_all_possible_costs()` eine Liste an allen möglichen neuen Blätter, die der Baum als nächstes hinzufügen kann. Das Blatt mit den geringsten Kosten wird dem Baum über den Aufruf `insert_new_node(obj)` hinzugefügt. Falls der Elternknoten des neuen Blatts selbst noch ein Blatt ist, entsprechen die Kosten des neuen Blatts den kombinierten Kosten für die ersten beiden Blätter. Das liegt daran, dass jeder innere Knoten mindesten zwei Kinder haben muss – daher stellen diese Kosten die effektiv neu entstehenden Kosten für den Baum dar. Das geschieht solange, bis der Baum genau so viele Blätter hat, wie es Buchstaben gibt. Seine Anzahl an Blättern erhält er über die Methode `get_size()`.

2.3.2 Permutation des Baums

Sobald ein balancierter Baum erstellt wurde, muss dieser optimiert werden. Dies geschieht über die Methode `find_best_tree`.

```
1 def find_best_tree(root: TreeNode) -> TreeNode:
```

Listing 4: Methode `find_best_tree`

Die Methode `find_best_tree` versucht, den übergebenen Baum so lang zu optimieren, bis alle Blätter als *protected* markiert sind und keine Blätter mehr hinzugefügt werden können. Dazu wird der Iterationsschritt über den Aufruf `improve_tree(root)` wiederholt und jeweils mit der Methode `calculate_tree(root)` die neuen Kosten des Baums berechnet. Sind die neuen Kosten ein neues Minimum wird der aktuelle Baum als bester Zwischenstand gespeichert. Können keine neuen Blätter dem Baum hinzugefügt werden und es tritt ein Fehler auf, wird der bis dahin beste gefundene Baum zurückgegeben.

```
1 def improve_tree(root: TreeNode) -> TreeNode:
```

Listing 5: Methode `imporve_tree`

Zu Beginn holt sich die Methode `improve_tree` alle Blätter im Baum über den Aufruf `get_leaves()`. Diese Liste wird dann aufsteigend nach den Kosten jedes Blatts sortiert. Der Elternknoten des Blatts mit den höchsten Kosten wird daraufhin selbst zu einem Blatt und verliert alle Kinder. Um die fehlenden Blätter wieder hinzuzufügen, wird noch die Methde 3 (`create_tree`) aufgerufen.


```
1 def calculate_tree(root: TreeNode) -> int:
```

Listing 6: Methode `calculate_tree`

Die Methode `calculate_tree` berechnet die gesamte Kosten des Baums. Dafür holt sie sich auch über den Aufruf `get_leaves()` alle Blätter im Baum. Die Liste an Blättern wird dann nach den Kosten aufsteigend sortiert und jedes Element wird wie in der Gleichung (3) berechnet und zusammenaddiert.

2.4 Laufzeitanalyse und Einordnung des Algorithmus

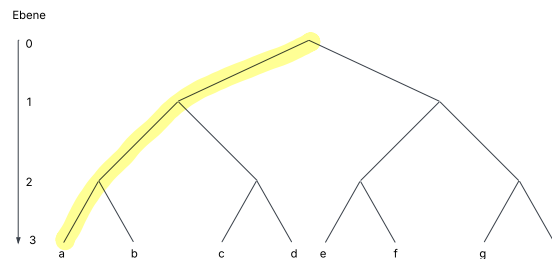


Abbildung 8: Balancierter Baum, Markierung der richtigen Knoten

2.4.1 Analyse des Algorithmus

Der Algorithmus endet, wenn keine Knoten mehr eingefügt werden können. Dies ist spätestens gegeben, wenn der Baum maximal ausgeartet ist (Abbildung 5). Allgemein kann jeder Knoten maximal $r - 1$ mal verschoben werden, bis er komplett auf der linken Seite ist. Dabei ist r die Anzahl an verschiedenen Perlen. Die Durchmesser sind hierbei unerheblich. Daraus folgt, dass die maximale Anzahl an Iterationen

$$I \leq (r - 1) \cdot \text{Anzahl falscher Knoten} \quad (5)$$

ist, bis zum maximal ausgearteten Baum. Falsche Knoten sind dabei innere Knoten, die beim ausbalancierten Baum noch nicht auf der Stelle des ausgearteten Baums sind, also alle bis auf die äußerst linken Knoten. (Abbildung 8)

Generell gilt, um die maximale Anzahl an inneren Knoten zu erhalten, sollte jeder Knoten möglichst wenig Kinder besitzen. Wie zuvor festgelegt, hat jeder innere Knoten mindestens zwei Kinder. Daraus folgt, dass die maximale Anzahl an inneren Knoten erreicht wird, wenn der Baum ein Binärbaum ist und kann durch

$$\text{innere Knoten} = w - 1 \quad (6)$$

berechnet werden.[7] Dabei ist w die Anzahl an Blättern/Codewörter. Auf jeder Ebene

existiert ein richtiger innere Knoten, deren Anzahl minimal mit

$$\text{richtige innere Knoten} = \log_r w \quad (7)$$

abgeschätzt werden kann.

Daher ergibt sich

$$I < (r - 1) \cdot (w - 1 - \log_r w) \quad (8)$$

Dies bedeutet eine Laufzeit von $O(r \cdot w)$.

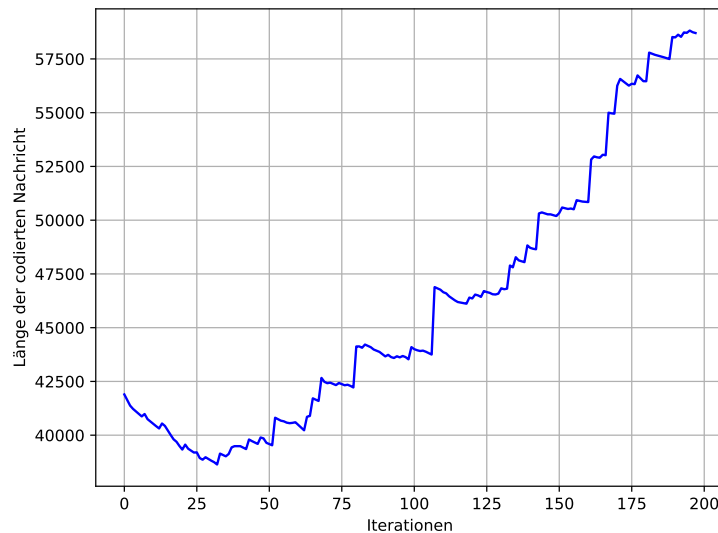


Abbildung 9: Länge der codierten Nachricht aufgetragen über mehrere Iterationen

Abbildung 9 zeigt wie die Länge der codierten Nachricht der Testfall *schmuck9* sich über die Iterationen hinweg verändert. Das Minimum wird bereits in der ersten Hälfte der Iterationen erreicht. Das zeigt gut, dass der Algorithmus kein Optimierungs-Algorithmus ist, sondern von einem Maximum zum anderen Maximum geht.

2.4.2 Stand der Forschung

Die vorliegende Fragestellung ist seit den 60iger-Jahren in der theoretischen Informatik bekannt. Dabei lieferte Huffman eine Lösung für den Spezialfall, von zwei Quellsymbolen.[6]

Für unseren Fall von mehr als zwei Quellsymbolen mit unterschiedlichen Kosten, entwickelte bereits Richard Karp 1961[5] eine Lösung. Diese wurde 1998 von Mordecai J. Golin und Günter Rote verbessert und ist bis heute das schnellste Verfahren, um auf eine exakte Lösung zu kommen.

”The minimum-cost prefix-free code for n words can be computed in $O(n^{C+2})$ time and $O(n^{C+1})$ space, if the costs of the letters are integers between 1 and C .”[4]

Für die Beispielaufgabe *schmuck5.txt* hat der Algorithmus eine Laufzeit 34^8 ($n=34$; $C=6$) Iterationen. Diese Art von Algorithmus kann wegen seiner langen Laufzeit nicht für die

vorgegebenen Beispielaufgabe verwendet werden.

Um eine polynominelle Laufzeit zu erreichen, entwickelten Mordecai J. Golin, Claire Mathieu und Neal E. Young im Jahre 2012 ein Approximationsverfahren.

Dieser Algorithmus basiert auf dem PTAS-Verfahren, wobei zuerst ein präfixfreier Code bis zu einer bestimmten Tiefe τ erstellt wird und dann die restlichen Codewörter durch ein Rundungsverfahren ergänzt werden. Dabei ist es zentral, dass die Wahrscheinlichkeitsverteilung der zu kodierenden Codewörter möglichst in großen Gruppen mit gleich großer Wahrscheinlichkeit eingeordnet werden können. [3] Zum Beispiel in Testfall 9 ist dies nicht gegeben, weshalb das Bestimmen des Vorbaums über 9^{24} Schritte benötigt. Somit ist dieses Verfahren für die gewählten Beispiele ebenfalls nicht verwendbar.

2.4.3 Fazit

Für die gegebene Fragestellung gibt es bislang weder ein Beweis, dass sie NP-schwer ist, als auch keinen Gegenbeweis, dass sie es nicht ist. Aktuell bekannte Verfahren für exakte Ergebnisse besitzen eine exponentielle Laufzeit. [4] Mein gefundener Algorithmus findet nicht die beste Lösung für das gegebene Problem. Er liefert jedoch in polynomieller Laufzeit eine gute Approximation und ist daher eine gute Abwägung zwischen Laufzeit und Genauigkeit. Somit wird nicht beantwortet, ob die Fragestellung NP-schwer ist.

2.5 Vergleich mit der Musterlösung

In der Musterlösung werden drei Lösungen vorgeschlagen. Der erste Ansatz ist ein Greedy-Algorithmus basierend auf Huffman. Die zweite Variante ist ein ILP-Ansatz nach Richard Karp und die dritte ist dessen Erweiterung von Mordecai J. Golin und Günter Rote. [2] Alle drei Ansätze waren mir bei Abgabe meiner Arbeit bekannt. Trotzdem entschied ich mich dagegen eines dieser Verfahren zu implementieren, vor allem aufgrund deren hohen Laufzeit, und entwickelte stattdessen meine eigenes Verfahren. Mein Algorithmus hat dabei eine durchschnittliche Abweichung von 1,27% vom Idealergebnis, besitzt jedoch im Gegensatz dazu eine sehr viel kürzere Laufzeit. [2] Für meine gute Arbeit erhielt ich daher einen Pluspunkt und erreichte insgesamt 21 Punkte für diese Aufgabe.

3 Aufgabe 2: Simultane Labyrinth

3.1 Einleitung

In dieser Aufgabe möchten drei Freunde, Anton, Bea und Chris, zwei Labyrinth gleichzeitig durchlaufen, die dieselben rechteckigen Maße (n, m) haben. Zwei Personen starten jeweils in ihrem Labyrinth bei der Position $(0, 0)$ und müssen das Ziel (n, m) erreichen. Die dritte Person gibt jeweils beiden immer dieselbe nächste Schrittanweisung. Wenn der nächste Schritt in einem Labyrinth durch eine Wand blockiert ist, bleibt die Person stehen. Gesucht ist eine möglichst kurze Abfolge von Schritten, die beide Labyrinth mit denselben Befehlen löst. Die ursprüngliche Aufgabe enthält normale Labyrinth. Im Spezialfall werden die Labyrinth um Gruben erweitert. Wenn ein Spieler in eine Grube fällt, wird seine Position auf die Startposition $(0, 0)$ zurückgesetzt. [1]

3.2 Theoretische Erklärung des Algorithmus

Zu Beginn wird für jedes Labyrinth eine Matrix erstellt (Abbildung 10), in der für jede Position ein Tupel aus einem Kostenwert und einer Richtung gespeichert wird. Die gespeicherte Richtung zeigt den optimalen Folgeschritt zum Ziel und ist in Tabelle 1 dargestellt.

Code	Richtung
0	Rechts
1	Links
2	Oben
3	Unten

Tabelle 1: Codierung der Richtungen im Labyrinth

Der Kostenwert cp gibt an, wie viele Schritte bis zum Ziel benötigt werden. In Abbildung 10 ist ein Labyrinth mit dazugehöriger Matrix abgebildet.

13;3	14;1	5;0	4;0	3;3
12;3	13;1	14;1	5;2	2;3
11;3	8;0	7;0	6;2	1;3
10;0	9;2	8;2	7;2	0;null

Abbildung 10: Labyrinth und Matrix

3.2.1 Erstellung der Matrix

Zu Beginn wird eine Matrix für jedes Labyrinth erstellt, deren Werte für alle Positionen zunächst leer sind. Ausgehend vom Zielpunkt durchläuft der Algorithmus rekursiv alle erreichbaren Nachbarpositionen. Für jeden Nachbarn wird in der Matrix ein Kostenwert eingetragen, der um eins höher ist als der Kostenwert des aktuellen Punkts, sowie die Richtung, aus der der Nachbar erreicht wurde.

3.2.2 Lösen der Labyrinth

Der Algorithmus entnimmt den momentan besten Weg aus der Liste der bisherigen Wege und erweitert diesen um einen weiteren Schritt, indem er in den Matrizen prüft, welchen Schritt die beiden Labyrinth für ihre jeweilige Position vorschlagen. Dieser Schritt wird an die bisherige Schrittfolge angehängt. Dabei entstehen in der Regel zwei unterschiedliche Wege, die zur Liste der bisherigen Wege hinzugefügt werden, es sei denn, beide Labyrinth schlagen denselben Schritt vor. Zur Auswertung der bisherigen Wege wird diese Rechnung angewandt:

$$c_b = cp_{s1} + cp_{s2} \quad (9)$$

$$c_a = cp_{a1} + cp_{a2} \quad (10)$$

$$c_w = \frac{c_b - c_a}{n} \quad (11)$$

wobei gilt:

- $s1$ und $s2$ sind die Kosten der Startposition

- $a1$ und $a2$ sind die Kosten der aktuellen Positionen

- c_w sind die Kosten des bisherigen Wegs

- n ist die Anzahl der benötigten Schritte

Der Bruch beschreibt die durchschnittliche Einsparung von Kosten pro Schritt. Je höher der Wert ist, desto effizienter ist der bisherige Pfad. Dabei ist 2 der maximal erreichbare Wert. Dieser ist nur bei identischen Labyrinth gegeben.

Der Weg mit der höchsten Einsparung gilt dabei als momentan bester Weg. Nach jeder Iteration wird das aktuelle Positionspaar $((x_1, y_1), (x_2, y_2))$ gespeichert, um zu verhindern, dass der Algorithmus exakt dasselbe Positionspaar noch einmal besucht. Zusätzlich gibt es auch die Begrenzung l , die festlegt, wie viele unterschiedliche Wege, sortiert nach ihren Kosten, in der Liste gespeichert werden dürfen. Diese Begrenzung beeinflusst maßgeblich die Laufzeit und die Genauigkeit des Algorithmus. In der Liste dürfen die Wege unterschiedliche Werte für n haben. Denn wichtig ist vor allem, dass ihre Kosten (Gleichung (11)) gering sind. Der gesamte Ablauf wird solange wiederholt, bis beide im Ziel sind.

Bei der Erweiterung des Algorithmus werden Gruben sowohl in der Kostenmatrix als auch beim Lösen wie normale Felder gesehen. Wenn einer der beiden Personen eine Grube berührt, wird seine Position auf die jeweilige Startposition zurückgesetzt. Daraus folgt, dass die Kosten der Grubenfelder den Kosten der Startpositionen entsprechen.

3.3 Implementierung des Algorithmus

3.3.1 Erstellung der Matrix

Zur Erstellung der Matrix wird die Klasse *Cost* benutzt.

Cost
List[int] horizontal_walls List[int] vertical_walls List[Tuple[int, int]] gruben int width int height List[List[Tuple[int, int]]] matrix
List[Tuple[int, int]] get_neighbors(int x, int y) void write_cost(int start_x, int start_y) List[List[Tuple[int, int]]] create_cost_matrix()

Abbildung 11: Klasse Cost

Für jedes Labyrinth wird ein Cost-Objekt erstellt und die Methode *cost_obj.create_cost_matrix()* aufgerufen. Das Cost-Objekt bekommt als Übergabewerte die horizontalen bzw vertikalen Wände sowie die Gruben und die Größe des Labyrinths.

```
1 def create_cost_matrix(self) -> list:
```

Listing 7: Methode *create_cost_matrix*

Die *create_cost_matrix* Methode erstellt zunächst eine leere Liste mit der Dimension [height][width], die als Kostenmatrix dient. Dann ruft sie die Methode *write_cost* auf, um die Kostenmatrix zu füllen.

```
1 def write_cost(self, start_x: int, start_y: int):
```

Listing 8: Methode *write_cost*

Zu Beginn der Methode *write_cost()* wird die Liste *stack* initialisiert. Sie speichert alle Felder, die noch besucht werden müssen. Jeder Eintrag in der Liste besitzt vier Werte: die x-Position, die y-Position, die aktuellen Kosten sowie die entgegengesetzte Richtung, aus welcher das Feld betreten wurde. Die Endposition wird als erstes Element hinzugefügt. Anschließend wird das erste Element der Liste entnommen und in die Kostenmatrix übertragen. Daraufhin werden alle benachbarten Felder mit der Methode *get_neighbours* ermittelt und dem *stack* hinzugefügt. Dabei werden die Kosten um eins erhöht und die jeweilige Richtung gespeichert. Das wird solange wiederholt, bis der *stack* leer ist. Falls ein Element schon in der Kostenmatrix ist, wird es übersprungen.

3.3.2 Lösen der Labyrinth

Zum Lösen der Labyrinth wird die Klasse *Solving* benutzt.

Solving
<pre> int width int height int cost_at_the_beginning List[Tuple[int, int]] gruben_1 List[Tuple[int, int]] gruben_2 List[int] horizontal_walls_1 List[int] vertical_walls_1 List[int] horizontal_walls_2 List[int] vertical_walls_2 List[List[Tuple[int, int]]] cost_matrix_1 List[List[Tuple[int, int]]] cost_matrix_2 dict visited </pre>
<pre> Tuple[int, int] next_position(Tuple[int, int] position, int next_move) int next_move(Tuple[int, int] position, List[List[Tuple[int, int]]] cost_matrix) int get_moving_cost(Tuple[int, int] position, List[List[Tuple[int, int]]] cost_matrix) int get_total_cost(Tuple[Tuple[int, int], Tuple[int, int]] positions) Move check_visited(Tuple[Tuple[int, int], Tuple[int, int]] next_position) List[Move] neighbours_cost(Move move) </pre>

Abbildung 12: Solving Klasse

Die Wege werden mit der Klasse *Move* dargestellt.

Move
<pre> List[int] moves Tuple[Tuple[int, int], Tuple[int, int]] positions int cost float weight </pre>

Abbildung 13: Move Klasse

Das Attribut *cost* beschreibt die aufsummierten Kosten der beiden aktuellen Positionen basierend auf den Werten aus den Kostmatrizen. Das Attribut *weight* wird gemäß der Gleichung (11) berechnet.

Zu Beginn wird ein *Solving*-Objekt initialisiert und eine leere Liste *moves_stack* angelegt, in der alle aktuellen Wege gespeichert werden. Anschließend wird ein erstes *Move*-Objekt erzeugt, dessen Startposition $((0, 0), (0, 0))$ ist und seine moves leer sind. In jedem Schritt wird das erste Element aus der Liste *moves_stack* entnommen. Dann wird mit folgendem Aufruf

```
1 new_moves = solving_obj.neighbours_cost(move)
```

eine Liste von möglichen Folgewege erzeugt. Die Liste wird mit den neuen Wegen erweitert, sortiert und auf die l besten Wege beschränkt.

```
1 def neighbours_cost(self, move: Move) -> list:
```

Listing 9: Methode `neighbours_cost`

Zu Beginn der Methode `neighbours_cost()` werden die beiden Positionen aus dem Attribut `positions` des übergebenen `move`-Objekt genommen. Anschließend werden mit den folgenden Aufrufen

```
1 move_1 = self.next_move(pos1, self.cost_matrix_1)
  move_2 = self.next_move(pos2, self.cost_matrix_2)
```

die jeweils nächsten möglichen Richtungen aus der Kostenmatrix bestimmt. Daraufhin werden die neuen zwei Positionspaare berechnet und überprüft, ob die beiden Positionspaare bereits besucht wurden.

Zum Schluss wird für jeden möglichen Zug ein `Move`-Objekt erstellt und zusammen in einer Liste übergeben. Falls einer der neuen Positionspaare bereits besucht wurde oder bereits im Ziel ist, wird der entsprechende Zug nicht übergeben (siehe Abbildung 17).

3.4 Laufzeitanalyse

Von jedem Positionspaar aus existieren im schlimmsten Fall zwei mögliche Schritte, entweder in Richtung des Ziels von Labyrinth 1 oder in Richtung des Ziels von Labyrinth 2. Das führt zu insgesamt $2^{(nm)^2}$ verschiedenen Möglichkeiten an Schritten. Aufgrund dieser großen Anzahl an Möglichkeiten untersucht der Algorithmus nicht alle Pfade vollständig, sondern macht nur eine Abschätzung. Der hier vorgestellte Ansatz garantiert daher nicht unbedingt den optimalen Lösungsweg, liefert jedoch in akzeptabler Laufzeit eine gute Annäherung. Der Algorithmus ähnelt einer Tiefensuche, da er einige Wege tief verfolgt, während andere nur kurz betrachtet werden.

Wie schon oben erwähnt beeinflusst die Begrenzung l , die Laufzeit und die Genauigkeit des Algorithmus stark.

Abbildung 14 zeigt die Anzahl der Schritte des gefundenen Lösungswegs in Abhängigkeit vom Parameter l für Testfall 4.

Auffällig ist, dass der Graph mit zunehmendem l gegen die Anzahl der Schritte des optimalen Wegs konvergiert. Ebenso erkennbar ist, dass der Graph nicht streng monoton fallend ist. Wenn die Steigung positiv ist, wird für ein größeren Wert von l ein schlechterer Pfad gefunden. Dies liegt daran, dass durch den höheren Wert von l neue Wege betrachtet werden. Diese erscheinen kurzfristig vorteilhafter als die bisherigen Wege und verdrängen dadurch die eigentlich besseren aus der Liste. Das führt insgesamt zu einem schlechteren Weg. Bei größeren l -Werten tritt dieser Effekt jedoch kaum noch auf, da die Liste groß

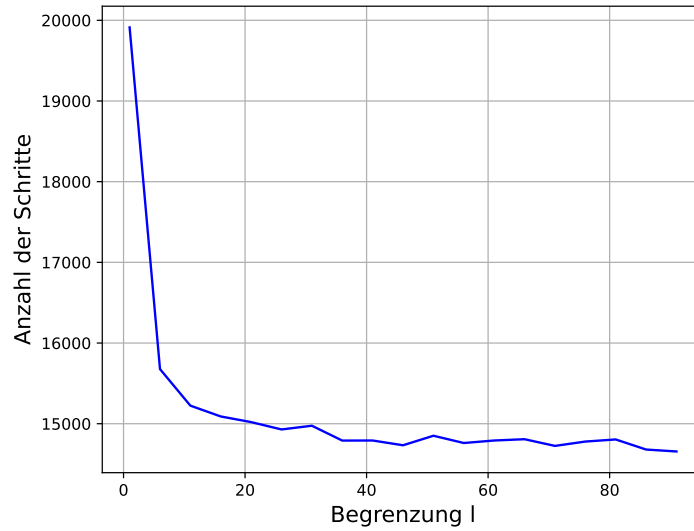


Abbildung 14: Anzahl der Schritte aufgetragen gegenüber der Begrenzung

genug ist, um alle relevanten Pfade zu behalten. Der Einfluss dieser Monotonieveränderungen wird daher mit wachsendem l vernachlässigbar.

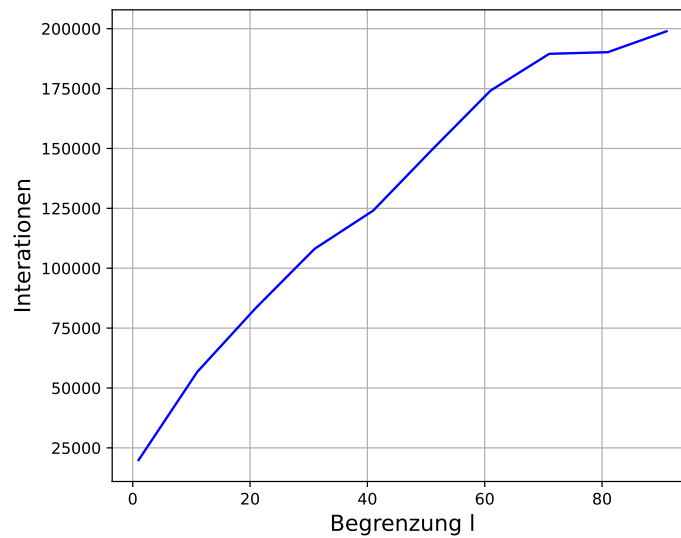


Abbildung 15: Anzahl der Iterationen gegenüber der Begrenzung

Abbildung 15 stellt die Anzahl an Iterationen dar, die zur Bestimmung des Lösungswegs benötigt werden in Abhängigkeit vom Parameter l .

Der Graph steigt mit wachsendem l stark an.

Allgemein kann die maximale Anzahl an Iterationen in Abhängigkeit von l wie folgt berechnet werden.

$$l \cdot (2c_1 + c_2) \quad (12)$$

wobei gilt:

- c_1 die Kosten vom Startpunkt aus der Kostenmatrix 1 sind
- c_2 die Kosten vom Startpunkt aus der Kostenmatrix 2 sind

Der Faktor $2c_1 + c_2$ beschreibt die maximale Anzahl an Schritten, aus denen der schlechteste Lösungsweg besteht. Zunächst erreicht Person1 das Ziel mit c_1 Schritten. Anschließend läuft Person2 denselben Weg zurück in ebenfalls c_1 Schritten und geht dann selber ins Ziel mit c_2 Schritten. Im schlechtesten Fall geht er für jeden Eintrag in der Liste, also l mal, die maximale Anzahl an Schritten. Daher ergibt sich die Gleichung (12), dies bedeutet, es ergibt sich eine Laufzeit von $o(l)$ ergibt. Praktisch gesehen ist die Laufzeit aber sehr viel kleiner als die maximale Laufzeit (Abbildung 16).

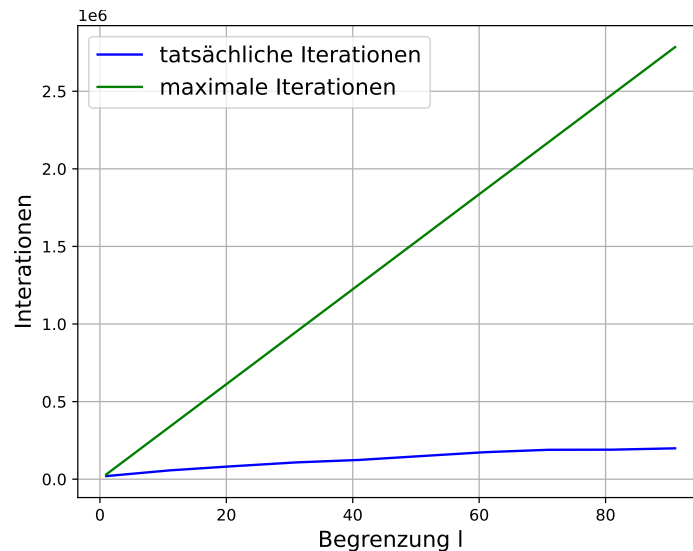


Abbildung 16: maximale Iterationen gegenüber tatsächliche Iterationen aufgetragen gegenüber der Begrenzung

Der grüne Graph zeigt die maximale Anzahl an Iterationen und der blaue die tatsächliche Anzahl an Iterationen.

Die Anzahl der Iterationen stehen in einem linearem Zusammenhang mit den Werten von l , während die Anzahl der Schritte bei größeren l -Werten gegen das optimale Ergebnis konvergieren. Je größer der Wert von l , desto mehr Iteration werden durchgeführt - und desto besser wird das Ergebnis. Die Genauigkeit lässt sich anhand des Parameters l jedoch nicht allgemein bestimmen, da sie stark von der Struktur des jeweiligen Labyrinths abhängt.

3.5 Vergleich mit der Musterlösung

In der Musterlösung werden zwei mögliche Lösungsstrategien vorgestellt. Die erste Strategie besteht in der Anwendung der Breitensuche auf zwei Labyrinth. Diese Variante wird jedoch in der Musterlösung aufgrund ihrer hohen Laufzeit als nicht praktikabel eingestuft. Die zweite Strategie basiert auf dem A-Star Algorithmus mit Priority Queue. Diese Vorgehensweise ähnelt stark meiner eigenen Idee. Wie in meiner Kostenmatrix wird auch hier für jede Position die Distanz zum Ziel berechnet.[2] Der wesentliche Unterschied liegt vor allem in dem effizienteren Warteschlangenmodell der Musterlösung.

Insgesamt lagen meine Ergebnisse, aus geschlossen von Testfall 8, bei 1,1% Abweichung vom Optimalwert.[2] Bei Testfall 8 fiel mein Ergebnis etwa 3-mal so groß aus, weshalb mir 2 Punkte im Kriterium "Verfahren mit guten Ergebnissen" abgezogen wurden. Außerdem war meine Überlegung zur Laufzeit des Verfahren nicht ausreichend detailliert, was zu einem weiteren Punktabzug führte.

Insgesamt erhielt ich für die Aufgabe 17 von 20 Punkten.

4 Fazit

1. kurz zusammenfassen was ich gemacht haben in aufgabe1 2. was für ergebnisse ich erzielt habe damit; wie viel punkte 3. kurz zusammenfassen was ich gemacht haben in aufgabe2 4. was für ergebnisse ich erzielt habe damit; wie viel punkte welcher paltz ich wurde was mit das gebracht habe warum ich dem wettbewerb gut finde Aufgrund des begrenzten Umfangs der Arbeit konnte ich keinen Beweis für die NP-Schwere erbrignen und eine präzisere Laufzeitanalyse durchführen. Dennoch konnte ich durch Vergleiche mit der Musterlösung Erkenntnisse über die Effizienz verschiedener Algorithmusstrategien gewinnen

Insgesamt konnte ich mit 38 von 40 Punkten den zweiten Platz erreichen und gehörte damit zu den 60 besten Teilnehmenden von über 250. Aufgrund dieser Leistung wurde ich für die Internationale Informatik-Olympiade vorgeschlagen und nehme derzeit an deren Trainingsprogramm teil. Durch diese Arbeit lernt ich Laufzeitanalysen selbstständig durchzuführen und mit wissenschaftlichen Texten zu arbeiten. Der Bwinf erweist sich insgesamt als gute Möglichkeit, Schüler an wissenschaftliches Arbeiten und algorithmisches Denken heranzuführen.

Der Bwinf eignet sich gut und sollte gefördert werden

5 Anhang

Im folgenden ist die Methode *neighbours_cost* als Flussdiagramm dargestellt.

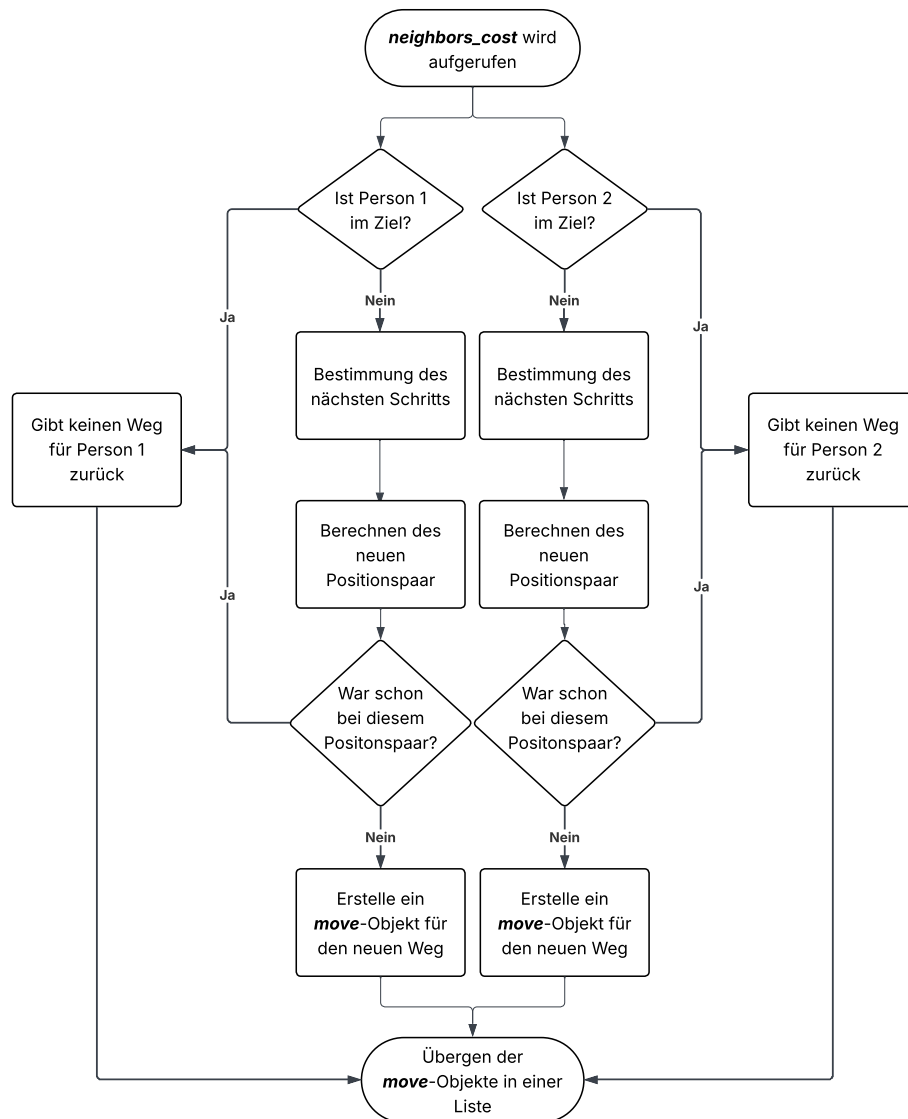


Abbildung 17: *neighbours_cost* Flussdiagramm

5.1 Lösungen der Testfälle

Das sind meine Ergebnisse der Testfälle im Vergleich zu der Musterlösung. Die vorgegebenen Testfälle können in Quelle 7 nachgeschaut werden und die optimalen Ergebnisse können in Quelle 6 eingesehen werden.

5.1.1 Aufgabe 1: Schmucknachrichten

Die Lösungen können in Tabelle 2 eingesehen werden.

Testfall	meine Gesamtkosten	optimalen Ergebnis
schmuck0.txt	114	113
schmuck00.txt	382	372
schmuck01.txt	1165	1150
schmuck1.txt	194	191
schmuck2.txt	135	135
schmuck3.txt	279	279
schmuck4.txt	137	137
schmuck5.txt	3189	3162
schmuck6.txt	234	234
schmuck7.txt	135409	134559
schmuck8.txt	3358	3287
schmuck9.txt	39635	36597

Tabelle 2: Lösungen der Aufgabe 1

5.1.2 Aufgabe 2: Simultane Labyrinth

Die Lösungen können in Tabelle 2 eingesehen werden.

Testfall	meine Gesamtkosten	optimalen Ergebnis
labyrinth0.txt	8	8
labyrinth1.txt	31	31
labyrinth2.txt	66	65
labyrinth3.txt	166	164
labyrinth4.txt	14497	14384
labyrinth5.txt	1345	1308
labyrinth6.txt	1844	1844
labyrinth7.txt	keine Lösung	keine Lösung
labyrinth8.txt	1433	472
labyrinth9.txt	1036	1012

Tabelle 3: Lösungen der Aufgabe 2

5.2 Quellcode

5.2.1 Aufgabe 1: Schmucknachrichten

```

class TreeNode:
2
    def __init__(self, cost, parent, perl_code):
4        self.perl_code = perl_code # Speichert die Codierung für den Knoten
        self.cost = cost
6        self.children = []
        self.parent = parent
8        self.is_leaf = True
        self.is_protected = False # Wenn True, dann kann er keine kinder mehr bekommen
10

```

```

def insert_new_node(self, obj):
12     if obj is self:
13         if self.is_leaf: # Damit jede parent immer mehr mindestens zwei kinder hat
14             self.is_leaf = False
15             treenode = TreeNode(self.cost + perl_costs[0], self, self.perl_code+[len
16             self.children.append(treenode)

18             treenode = TreeNode(self.cost + perl_costs[len(self.children)], self, self.p
19             self.children.append()
20             return

22     for child in self.children:
23         child.insert_new_node(obj)
24
25 def get_size(self) -> int:
26     if self.is_leaf:
27         return 1
28     return sum(child.get_size() for child in self.children)

30 def get_leaves(self) -> list:
31     if self.is_leaf:
32         return [self]

34     children_leaves = []
35     for child in self.children:
36         for i in child.get_leaves():
37             children_leaves.append(i)
38     return children_leaves

40 def get_all_possible_costs(self) -> dict:
41     children_cost = {}
42     if len(self.children) < len(perl_costs) and self.is_protected is False:
43         if self.is_leaf:
44             children_cost[self] = self.cost + perl_costs[0] + self.cost + perl_costs
45         else:
46             children_cost[self] = self.cost + perl_costs[len(self.children)]

48     for child in self.children:
49         children_cost.update(child.get_all_possible_costs())
50     return children_cost

52 def get_internal_nodes(self):
53     if self.is_leaf:
54         return 0
55     return sum(child.get_internal_nodes() for child in self.children) + 1

1 def create_tree(root: TreeNode) -> TreeNode:
2     while root.get_size() < len(distribution):
3         possible_children = root.get_all_possible_costs()

```

```

        smallest_value = min(possible_children.items(), key=lambda item: item[1])
5
        root.insert_new_node(smallest_value[0])
7    return root

9
def calculate_tree(root):
11    all_leaves = root.get_leaves()
    all_leaves = sorted(all_leaves, key=lambda x: (x.cost, len(x.parent.children)))
13    total_count = 0
    for i in range(len(all_leaves)):
15        total_count += all_leaves[i].cost * distribution[i]
    return total_count
17

19 def improve_tree(root: TreeNode) -> TreeNode:
    all_leaves = root.get_leaves()
21    all_leaves = sorted(all_leaves, key=lambda x: (x.cost, len(x.parent.children)))
    highest_cost = 0
23    parent_node = None
    for i in range(len(all_leaves)):
25        if all_leaves[i].parent.parent is not None:
            if all_leaves[i].cost * distribution[i] > highest_cost:
27                highest_cost = all_leaves[i].cost * distribution[i]
                parent_node = all_leaves[i].parent
29
    parent_node.is_leaf = True
31    parent_node.children = []
    parent_node.is_protected = True
33
    return create_tree(root)
35

37 def find_best_tree(root):
    best_value = calculate_tree(root)
39    best_root = copy.deepcopy(root)
    while True:
41        try:
            root = improve_tree(root)
43            new_calc = calculate_tree(root)

            if new_calc < best_value:
                best_value = new_calc
                best_root = copy.deepcopy(root)
47        except Exception as e:
49            # Gibt keine Verbesserung mehr
            return best_root
51

53 def output(root):
    print(f"Gesamtkosten: {calculate_tree(root)}")

```

```

55 all_leaves = root.get_leaves()
    all_leaves = sorted(all_leaves, key=lambda x: (x.cost, len(x.parent.children)))
57 for i in range(len(all_leaves)):
    print(f"{characters[i]}: {all_leaves[i].perl_code}")
59

61 def main():
    # Initiiere Wurzel objekt
63 root = TreeNode(0, None, [])

65 # Erstellen des balancierten Baums
    root = create_tree(root)
67
    # Verbessern des Baums
69 root = find_best_tree(root)
    output(root)

```

5.2.2 Aufgabe 2: Simultane Labyrinth

```

1 class Move:

3     def __init__(self, moves, positions, cost, heights_cost, weight=None):
        self.moves = moves
5         self.positions = positions
        self.cost = cost
7         self.weight = weight if weight is not None else (heights_cost - cost) / len(moves)

    class Solving:
2     def __init__(self, cost_matrix_1, cost_matrix_2, vertical_walls_1, horizontal_walls_1,
        vertical_walls_2, horizontal_walls_2, gruben_1, gruben_2, width, height):
4         self.width = width
        self.height = height
6         self.cost_at_beginning = cost_matrix_1[0][0][0] + cost_matrix_2[0][0][0]

8         self.gruben_1 = gruben_1
        self.gruben_2 = gruben_2
10
        self.vertical_walls_1 = vertical_walls_1
12        self.horizontal_walls_1 = horizontal_walls_1

14        self.vertical_walls_2 = vertical_walls_2
        self.horizontal_walls_2 = horizontal_walls_2
16
        self.cost_matrix_1 = cost_matrix_1
18        self.cost_matrix_2 = cost_matrix_2

20        self.visited = {}

```



```

# Funktionen und Bewegungsdeltas nur einmal definieren
22 self.move_funcs = [test_right, test_left, test_up, test_down]
    self.move_deltas = [(1, 0), (-1, 0), (0, -1), (0, 1)]

24
def next_position(self, position, next_move):
26     mf = self.move_funcs
    md = self.move_deltas
28     pos1 = position[0]
    pos2 = position[1]
30
    # Für die erste Position
32     if pos1 != (self.width - 1, self.height - 1):
        matrix1 = self.vertical_walls_1 if next_move < 2 else self.horizontal_walls_1
34         if mf[next_move](pos1[0], pos1[1], matrix1):
            pos1 = (pos1[0] + md[next_move][0], pos1[1] + md[next_move][1])
36         if pos1 in self.gruben_1:
            pos1 = (0, 0)
38
    # Für die zweite Position
40     if pos2 != (self.width - 1, self.height - 1):
        matrix2 = self.vertical_walls_2 if next_move < 2 else self.horizontal_walls_2
42         if mf[next_move](pos2[0], pos2[1], matrix2):
            pos2 = (pos2[0] + md[next_move][0], pos2[1] + md[next_move][1])
44         if pos2 in self.gruben_2:
            pos2 = (0, 0)
46     return pos1, pos2

48 def next_move(self, pos, cost_matrix):
    w = self.width
50     h = self.height
    return None if pos == (w - 1, h - 1) else cost_matrix[pos[1]][pos[0]][1]
52
def get_moving_cost(self, pos, cost_matrix):
54     w = self.width
    h = self.height
56     return 0 if pos == (w - 1, h - 1) else cost_matrix[pos[1]][pos[0]][0]

58 def get_total_cost(self, positions):
    pos1, pos2 = positions
60     return self.get_moving_cost(pos1, self.cost_matrix_1) + self.get_moving_cost(pos2,

62 def check_visited(self, next_position, length, move):
    # Erstelle einen Schlüssel als Tupel, das beide Positionen enthält
64     key = (next_position[0][0], next_position[0][1],
            next_position[1][0], next_position[1][1])
66     if key in self.visited and self.visited[key] <= length:
        return None
68     self.visited[key] = length
    return move
70
def neighbours_cost(self, move: Move):

```

```

72     # Berechne beide Positionen anhand der Bewegungsfolge
    pos1 = move.positions[0]
74     pos2 = move.positions[1]

76     # Ermittle den nächsten Zug für beide Positionen
    move_1 = self.next_move(pos1, self.cost_matrix_1)
78     move_2 = self.next_move(pos2, self.cost_matrix_2)

80     # Nächsten Positionspaare berechnen
    next_position_1 = self.next_position(move.positions, move_1) if move_1 is not None
82     next_position_2 = self.next_position(move.positions, move_2) if move_2 is not None

84     # Testen ob die Positionspaare schon einmal da waren
    move_1 = self.check_visited(next_position_1, len(move.moves) + 1, move_1) if move_1
86     move_2 = self.check_visited(next_position_2, len(move.moves) + 1, move_2) if move_2

88     results = []
    if move_1 is not None:
90         results.append(Move(move.moves + [move_1], next_position_1, self.get_total_cost(
            self.cost_at_beginning)))
92     if move_2 is not None:
        results.append(Move(move.moves + [move_2], next_position_2, self.get_total_cost(
94             self.cost_at_beginning)))

    return results
96

1 class Cost:

3     def write_cost(self, start_x, start_y):
        stack = [(start_x, start_y, 0, None)]

5         while stack:
7             x, y, cost, direction = stack.pop()
            if self.matrix[y][x] != 0:
9                 continue

11            self.matrix[y][x] = (cost, direction)

13

            a = self.get_neighbors(x, y)
15            for neighbor in a:
                nx, ny, ndirection = neighbor
17                if self.matrix[ny][nx] == 0:
                    stack.append((nx, ny, cost + 1, ndirection))
19

21    def create_cost_matrix(self):
        self.matrix = [[0 for _ in range(self.width)] for _ in range(self.height)]
23        self.write_cost(self.width - 1, self.height - 1)

```

```

        return self.matrix

25

def create_matrix(data, height):
2   vertical_walls = [data.pop(0) for _ in range(height)]
   horizontal_walls = [data.pop(0) for _ in range(height - 1)]
4   gruben_anzahl = int(data.pop(0)[0])
   gruben = [tuple(data.pop(0)) for _ in range(gruben_anzahl)]
6   return horizontal_walls, vertical_walls, gruben

8
def create_matrix_for_laby(horizontal_walls, vertical_walls, gruben, width, height):
10  cost_obj = Cost(horizontal_walls, vertical_walls, gruben, width, height)
   return cost_obj.create_cost_matrix()
12

14 def moving(cost_matrix_1, cost_matrix_2, width, height, horizontal_walls_1, vertical_walls_1,
            vertical_walls_2, gruben_1, gruben_2, l):
16

18  solving_obj = Solving(cost_matrix_1, cost_matrix_2, vertical_walls_1, horizontal_walls_1,
            horizontal_walls_2, gruben_1, gruben_2, width, height)
20

   moves = [Move([], ((0, 0), (0, 0)), 0, heights_cost=0, weight=0)]
22   while True:

24       # Erstes Element aus der Liste nehmen
       move = moves[0]
26       moves.pop(0)

28       # Liste mit den neuen Wegen erweitern
       new_moves = solving_obj.neighbours_cost(move)
30       moves.extend(new_moves)

32       moves = sorted(moves, key=lambda obj: (-obj.weight, len(obj.moves)))
       moves = moves[:1] # Nur die 1 besten Züge speichern
34

       if moves[0].cost == 0:
36           return moves[0]

38
def output(move):
40   print(len(move.moves))
   print(move.moves)
42

44 def main():
   # Datei Einlesen
46   width, height, data = read_data_from_file('input/labyrinthe8.txt')

```

```

48  # Listen erstellen
    horizontal_walls_1, vertical_walls_1, gruben_1 = create_matrix(data, height)
50  horizontal_walls_2, vertical_walls_2, gruben_2 = create_matrix(data, height)

52  # Matrix für beide Labyrinth erstellen
    cost_matrix_1 = create_matrix_for_laby(horizontal_walls_1, vertical_walls_1, gruben_1,
54  cost_matrix_2 = create_matrix_for_laby(horizontal_walls_2, vertical_walls_2, gruben_2,

56  # Weg finden

58  # TODO: Wert für l anpassen
    l = 100
60  try:
        move = moving(cost_matrix_1, cost_matrix_2, width, height, horizontal_walls_1, ver
62                                horizontal_walls_2, vertical_walls_2, gruben_1, gruben_2, l)
        output(move)

64
    except:
66        print("No solution found")

68
    if __name__ == "__main__":
70        main()

```

5.3 Quellen

- [1] *Bundeswettbewerb Informatik 43.2 Runde – Aufgaben*. Aufgerufen am 11.10.2025. 2025. URL: https://bwinf.de/fileadmin/wettbewerbe/bundeswettbewerb/43/2_runde/Aufgaben432.pdf.
- [2] *Bundeswettbewerb Informatik 43.2 Runde – Lösungshinweise*. Aufgerufen am 11.10.2025. 2025. URL: https://bwinf.de/fileadmin/wettbewerbe/bundeswettbewerb/43/2_runde/Loesungshinweise432.pdf.
- [3] Mordecai J. Golin, Claire Mathieu und Neal E. Young. „Huffman Coding with Letter Costs: A Linear-Time Approximation Scheme“. In: *SIAM Journal on Computing* 41.3 (2012), S. 684–713.
- [4] Mordecai J. Golin und Günther Rote. „A Dynamic Programming Algorithm for Constructing Optimal Prefix-Free Codes with Unequal Letter Costs“. In: *IEEE Transactions on Information Theory* 44.5 (Sep. 1998).
- [5] R. Karp. „Minimum-Redundancy Coding for the Discrete Noiseless Channel“. In: *IRE Transactions on Information Theory* 7.1 (Jan. 1961).
- [6] Justus Piater. *Algorithmen und Datenstrukturen*. Vorlesungsunterlagen, Seiten 4–13, Stand 29.05.2024. 2024.

- [7] Ralph-Hardo Schulz. *Codierungstheorie: Eine Einführung*. 2. Aufl. 2018. ISBN: 978-3-322-80328-3.